

MANUAL INTERACTIVO · JAVASCRIPT

De la lógica de PSeInt al código real en JavaScript

prompt() · alert() · 100% archivos .js · sin una línea de HTML

Sublime Text + Consola del navegador



¡Bienvenido/a!

No estás empezando de cero: ya sabés pensar como programador/a. Lo que cambia acá es la sintaxis, no la lógica.

01

Entorno y metodología

Sublime Text + Consola del navegador

02

Equivalencias PSeInt ↔ JS

La misma lógica, otra sintaxis

03

Variables y prompt()

let, const y conversión de tipos

04

Control de flujo

if, switch y while con prompt()

05

Funciones

function, parámetros y return

06

Proyecto integrador

Un programa completo paso a paso

07

Desafíos prácticos

3 ejercicios para resolver solo/a

¿Por qué Sublime Text?

En PSeInt escribías y ejecutabas en el mismo lugar. Ahora esas dos tareas se separan: Sublime Text edita, el navegador ejecuta.

1

Liviano y rápido

Abre instantáneo, sin configuración compleja para empezar a escribir código.

2

Resaltado de sintaxis

Colorea automáticamente palabras clave, textos y comentarios: leer código se vuelve más fácil.

3

Sin distracciones

Una interfaz simple, enfocada solo en escribir y organizar tus archivos .js.

Idea clave: Sublime Text = donde escribo el código · Navegador = donde lo ejecuto

Una aclaración antes de arrancar



Un .js NO se ejecuta solo

Por seguridad, los navegadores solo ejecutan JavaScript vinculado a un documento HTML. Si abris un .js suelto con doble clic, el navegador muestra el código como texto plano — ningún `prompt()` ni `alert()` va a aparecer.



La solución real: la Consola

Existe una forma de trabajar exclusivamente con archivos .js, sin escribir una sola línea de HTML: la Consola del navegador (DevTools). Ahí pegamos y ejecutamos todo nuestro código durante este manual.

F12 → **pestaña "Console"** → **pegar código** → **Enter**

Chrome puede pedir escribir "allow pasting" la primera vez: es normal.

Flujo de trabajo, paso a paso



🔄 Ciclo que vas a repetir todo el curso:

Editar en Sublime → Guardar → Copiar todo → Pegar en la Consola → Enter → Ver el resultado

Opción 2 · Abrir con el navegador

La Consola no es la única forma. Con un archivo “loader” que armás una sola vez, después simplemente hacés doble clic para ejecutar.

Opción 1 · Consola (ya vista)

Opción 2 · Abrir con el navegador

```
ejecutar.html
<!DOCTYPE html>
<html>
<body>
  <script src="saludo.js"></script>
</body>
</html>
```

Este archivo se crea UNA sola vez y nunca se edita. Solo cambiás el nombre del .js del src según el ejercicio.

Cada vez que querés probar:

- 1 Editá y guardá tu .js en Sublime Text.
- 2 Doble clic en ejecutar.html (o F5 si ya está abierto).
- 3 Los prompt() y alert() aparecen apenas carga la página.

💡 Recordá: un .js suelto sigue sin ejecutarse solo. Este loader es lo que hace que “abrir con el navegador” funcione de verdad.

Ejemplo · Saludo con nombre y edad

El mismo archivo .js, probado de las dos formas que ya conocés

```
saludo.js

// 1. Pedimos el nombre del usuario
let nombre = prompt("¿Cuál es tu nombre?");

// 2. Pedimos la edad (llega como texto: hay que convertirla)
let edad = Number(prompt("¿Cuántos años tenés?"));

// 3. Armamos el saludo combinando texto y número
alert("¡Hola " + nombre + "! Tenés " + edad + " años. ¡Un gusto saludarte!");
```

Probalo de las 2 formas

 Por Consola

Copió todo (Ctrl+A, Ctrl+C) y pegalo en la Consola (F12). Enter.

 Por archivo

Con ejecutar.html apuntando a saludo.js, doble clic sobre ejecutar.html.

Equivalencias: PSeInt vs. JavaScript

La misma lógica que ya conocés. Solo cambian las palabras y los símbolos.

Acción	PSeInt	JavaScript
Definir variable	Definir nombre Como Cadena	<code>let nombre;</code>
Leer dato	Leer nombre	<code>let nombre = prompt("...");</code>
Mostrar en pantalla	Escribir "Hola", nombre	<code>alert("Hola " + nombre);</code>
Condicional	Si ... Entonces ... FinSi	<code>if (...) { ... }</code>
Selección múltiple	Según opcion Hacer ... FinSegun	<code>switch (opcion) { case: ... break; }</code>
Bucle mientras	Mientras cond Hacer ... FinMientras	<code>while (cond) { ... }</code>
Función	Funcion r = Sumar(a,b) ... FinFuncion	<code>function sumar(a,b){ return a+b; }</code>



MÓDULO 01

Variables y entrada de datos

let, const, prompt() y alert() — la base de todo programa interactivo

Declarar variables y hablar con el usuario

let y const

```
let edad = 25;    // PUEDE cambiar
const PI = 3.1416; // NO puede cambiar
```

💡 Regla práctica: si vas a leer un dato con `prompt()` y guardarlo, usá `let`. Si es un valor fijo, usá `const`.

A diferencia de PSeInt, no hace falta declarar el tipo (Cadena, Entero): JS lo detecta solo (tipado dinámico).

prompt() y alert()

prompt("mensaje")

Pide un dato al usuario. Equivale a Leer.

alert("mensaje")

Muestra un mensaje. Equivale a Escribir.

```
let nombre = prompt("¿Cómo te llamas?");
alert("Bienvenido/a, " + nombre);
```

⚠️ prompt() siempre devuelve texto

El detalle más importante de este módulo — y la trampa #1 de todo principiante.

problema.js

```
let n1 = prompt("Primer número:"); // "5"  
let n2 = prompt("Segundo número:"); // "3"  
  
alert(n1 + n2); // ¡"53"! Se pegan los textos
```

solucion.js

```
let n1 = Number(prompt("Primer número:")); // (ej 3)  
let n2 = Number(prompt("Segundo número:")); // (ej 5)  
  
alert(n1 + n2); // 8 -> ¡correcto!
```

Función	Uso	Ejemplo
Number()	Entero o decimal	Number("3.5") → 3.5
parseInt()	Entero (descarta decimales)	parseInt("3.5") → 3
parseFloat()	Decimal explícito	parseFloat("3.5") → 3.5

En PSeInt esto era invisible: el intérprete lo resolvía solo. En JS, vos sos responsable de convertir el dato.

Ejemplo 1 · Saludo personalizado

ejemplo1.js

```
// 1. Pedimos el nombre. prompt() devuelve texto (String).
let nombre = prompt("¿Cuál es tu nombre?");

// 2. Pedimos la edad (también llega como texto).
let edadTexto = prompt("¿Cuántos años tenés?");

// 3. Convertimos la edad a número para poder operar.
let edad = Number(edadTexto);

// 4. Calculamos el año de nacimiento aproximado.
let anioNacimiento = 2026 - edad;

// 5. Mostramos el resultado (+ concatena texto y número).
alert("Hola " + nombre + ", naciste en " + anioNacimiento);
```

💡 Para probar

Guardá el archivo, copió todo (Ctrl+A, Ctrl+C) y pegalo en la Consola del navegador (F12) para verlo funcionar.

Notá el uso de + para unir textos y números en un solo mensaje.

Ejemplo 2 · Suma: incorrecta vs. correcta

ejemplo2.js

```
// --- Versión INCORRECTA (solo para comparar) ---
let a = prompt("Ingresá el primer número:");
let b = prompt("Ingresá el segundo número:");
// alert(a + b); -> concatenaría: "5" + "3" = "53"

// --- Versión CORRECTA ---
// --- Opcion 1
let a = Number(prompt("Ingresá el primer número:"));
let b = Number(prompt("Ingresá el segundo número:"));

// --- Opcion 2
let numA = Number(a); // convertimos a número
let numB = Number(b); // ídem

let suma = numA + numB; // ahora sí es matemática

alert("La suma es: " + suma);
```

⚠ Trampa común

Comparar ambas versiones ayuda a visualizar el error de concatenación antes de aplicar la solución.

Es el error #1 de quienes recién llegan de PSeInt.



MÓDULO 02

Estructuras de control de flujo

if / else, switch y while — decisiones y repeticiones con prompt()

Condicionales y operadores lógicos

condicionales.js

```
let edad = Number(prompt("Ingresá tu edad:"));

if (edad >= 18) {
    alert("Sos mayor de edad.");
} else if (edad >= 13) {
    alert("Sos adolescente.");
} else {
    alert("Sos menor de edad.");
}
```

PSeInt	JS	Significado
Y	&&	ambas verdaderas
O		al menos una
No	!	negación

🐾 Para comparar valores se usa === (triple igual), no un solo =, que es de asignación.

logicos.js

```
if (edad >= 18 && entrada === "si") {
    alert("Podés ingresar.");
} else { alert("No cumplís."); }
```

Selección múltiple: switch

Equivale a Según...Hacer...FinSegun

switch-dias.js

```
let dia = Number(prompt("Día (1 al 7):"));
let nombreDia;

switch (dia) {
  case 1: nombreDia = "Lunes"; break;
  case 2: nombreDia = "Martes"; break;
  // ... casos 3 a 6 ...
  case 7: nombreDia = "Domingo"; break;
  default: nombreDia = "Inválido";
}

alert("Día: " + nombreDia);
```

⚠️ La importancia del break

En PSeInt cada opción de Según se evalúa de forma independiente: corta automáticamente.

En JS NO es automático. Sin break, el código sigue ejecutando los case siguientes aunque no coincidan ("fall-through").

El break le dice a JS: "ya encontré la opción correcta, salí del switch".

Bucle while para validar entradas

Equivalencia a Mientras...Hacer...FinMientras — ideal para repetir una pregunta hasta recibir un dato válido

```
validacion-while.js

let numero = Number(prompt("Ingresá un número mayor a 0:"));

while (numero <= 0) {
    alert("Valor incorrecto. Debe ser mayor a 0.");
    numero = Number(prompt("Ingresá un número mayor a 0:"));
}

alert("¡Gracias! Ingresaste: " + numero);
```

💡 Patrón clave

Se pregunta una vez fuera del bucle, y dentro del while se vuelve a preguntar si la condición de error se sigue cumpliendo.

Es el mismo patrón de validación que ya usabas en PSeInt.



MÓDULO 03

Funciones

function, parámetros y return — reutilizando lógica con prompt() y alert()

Declarar una función y usar return

```
funcion.js  
  
function sumar(a, b) {  
    let resultado = a + b;  
    return resultado;  
}
```

function

Palabra clave para declarar (como Funcion en PSeInt).

(a, b)

Parámetros que recibe (entradas de la función).

return

Devuelve un valor a quien la llamó (como resultado = en PSeInt).

El patrón que usamos siempre

1. prompt() por fuera de la función
2. Los datos se pasan como argumentos
3. La función calcula y hace return
4. alert() muestra lo que devolvió

```
function sumar(a, b) { return a + b; }  
  
let n1 = Number(prompt("N1:"));  
let n2 = Number(prompt("N2:"));  
alert("Resultado: " + sumar(n1, n2));
```

Dos ejemplos progresivos

area-rectangulo.js

```
// Recibe base y altura, devuelve el área
function calcularArea(base, altura) {
    return base * altura;
}

let base = Number(prompt("Base:"));
let altura = Number(prompt("Altura:"));

let area = calcularArea(base, altura);

alert("El área es: " + area);
```

par-impar.js

```
// Recibe un número, devuelve un texto
function esParOImpar(numero) {
    if (numero % 2 === 0) {
        return "par";
    } else {
        return "impar";
    }
}

let n = Number(prompt("Número:"));
let tipo = esParOImpar(n);
alert("El número " + n + " es " + tipo);
```



PROYECTO

Proyecto integrador

Una caja de ahorro con menú, `prompt()`, `while`, `switch`, funciones y `alert()` — todo junto

Objetivo: menú de caja de ahorro

Un único archivo caja-de-ahorro.js que combina todo lo aprendido. Lo vamos a construir en 3 etapas: probando en la Consola después de cada una.

prompt()

Leer opciones y montos

while

Repetir el menú hasta elegir
Salir

switch

Dirigir el flujo según la opción

función()

Depositar, retirar, consultar

alert()

Comunicar cada resultado

Paso A

Estructura base

El menú con while + switch, todavía con alert() de prueba.

Paso B

Lógica con funciones

Reemplazamos las pruebas por funciones reales que actualizan el saldo.

Paso C

Validaciones

Agregamos while internos para evitar montos inválidos.

Paso A → B: del esqueleto a las funciones

caja-de-ahorro.js - Paso A

```
let opcion = "0";

while (opcion !== "4") {
  opcion = prompt(
    "1.Saldo 2.Depositar 3.Retirar 4.Salir"
  );

  switch (opcion) {
    case "1": alert("Consultar saldo"); break;
    case "2": alert("Depositar"); break;
    case "3": alert("Retirar"); break;
    case "4": alert("Saliendo..."); break;
    default: alert("Opción inválida");
  }
}
```

caja-de-ahorro.js - Paso B

```
let saldo = 0;

function depositar(saldoActual) {
  let monto = Number(prompt("¿Cuánto?"));
  let nuevoSaldo = saldoActual + monto;
  alert("Nuevo saldo: $" + nuevoSaldo);
  return nuevoSaldo;
}

// retirar() y consultarSaldo() son similares

// dentro del switch:
// case "2": saldo = depositar(saldo); break;
```

Paso C · Repetición y validaciones con while

El mismo patrón de validación del Módulo 2, ahora dentro de una función

```
caja-de-ahorro.js - Paso C (final)

function retirar(saldoActual) {
  let monto = Number(prompt("¿Cuánto querés retirar?"));

  // Validación: no negativo ni mayor al saldo
  while (monto <= 0 || monto > saldoActual) {
    alert("Monto inválido. Máximo: $" + saldoActual);
    monto = Number(prompt("¿Cuánto querés retirar?"));
  }

  let nuevoSaldo = saldoActual - monto;
  alert("Retiro realizado. Nuevo saldo: $" + nuevoSaldo);
  return nuevoSaldo;
}

// depositar() se valida de forma equivalente (monto > 0)
```

✓ Resultado

Con esto tenés un programa completo en un único archivo .js, sin usar HTML en ningún momento: entrada de datos, control de flujo, funciones y validación.

¡A programar!

Ya tenés todo lo necesario para escribir programas interactivos en JavaScript usando `prompt()` y `alert()`, editando en Sublime Text y ejecutando directamente en la Consola del navegador.

 **El ciclo de trabajo, siempre:**

Escribir → Guardar → Copiar todo → Pegar en la Consola → Enter